CHAPTER 2

Your Python Toolkit and Environment Setup

Chapter at a glance

Before you start: Create folders and run basic **terminal commands**.
By the end, you can: organize data and notebooks, configure an environment, install dependencies, and run labs reproducibly.

2.1 The course project layout

Use `UTP-MACHINE-LEARNING/` as the project root, with these folders:

```
all_datasets/
  ames_housing_dataset.csv
  bike_sharing_dataset_day.csv
  bike_sharing_dataset_hour.csv
  breast_cancer_dataset.csv
  credit_card_fraud_dataset.csv
  heart_failure_dataset.csv
  mall_customers_dataset.csv
  medical_cost_personal_dataset.csv
  optical_recognition_handwritten_digits.zip
  red_wine_quality_dataset.csv
  student_performance_dataset.csv
  titanic_dataset.csv
```

```
all_experiments/
  lab01_features_worked.ipynb
  lab02_supervised_workflow_worked.ipynb
  lab03_supervised_models_worked.ipynb
  lab04_clustering_pca_worked.ipynb
  lab05_anomaly_limited_labels_worked.ipynb
  lab06_reinforcement_worked.ipynb
  lab07_deep_learning_cnn_worked.ipynb
  lab08_deep_learning_lstm_worked.ipynb
  predict_passenger.py
models/
results/lab01/ ... lab08/
requirements.txt
```

Keep the digits ZIP intact. Each lab has one worked notebook; `predict_passenger.py` uses the saved Lab 2 model. Notebooks write models and cards to `models/` and evidence to `results/`; do not edit generated outputs.

2.2 Download the datasets

Click a **dataset name** to download it, then extract the files into **`all_datasets/`**.

Kaggle Dataset	Description
Breast Cancer Wisconsin	Lab 1: scaling; Lab 2: thresholds; Lab 3: KNN, naive Bayes, SVM; Lab 5: limited labels
Medical Cost Personal Data	Lab 1: pipelines; Lab 2: linear and ridge regression
Student Performance	Lab 1: data quality, prediction timing, feature selection
Titanic Dataset	Lab 1: feature engineering; Lab 2: baselines, tuning, saving; Lab 3: trees
Bike Sharing	Lab 2: daily/hourly time splits; Lab 8: LSTM forecasting
Heart Failure Prediction	Lab 2 Try-it: mixed-type classification
Ames Housing	Lab 3: random forests and gradient boosting
Red Wine Quality	Lab 3: ensembles and feature importance
Mall Customers	Lab 4: K-means and DBSCAN
Optical Recognition of Handwritten Digits	Lab 4: PCA; Lab 7: CNNs
Credit Card Fraud Detection	Lab 5: anomaly detection and imbalanced evaluation

2.2.1 Download steps

Step 1: Open the link, click **Download**, and **extract** the archive.

Step 2: Copy the required **CSV files or images** into **`all_datasets/`**.

Step 3: Check that your notebook's **file paths** match the filenames, including **capitalization**.

Common mistake

For **`FileNotFoundError`**, check the working directory and filename. Fix the path rather than moving files into the project root.

2.3 Two ways to set up: plain Python or Anaconda

Key idea

Use one environment per project; activate it before installing or running packages.

2.4 Route A: install Python and configure PATH

Download Python from the **official downloads page**.

Windows	Run the installer, enable **`PATH`**, and reopen the terminal. Use **`py`**.
macOS	Run the **`.pkg` installer** and restart Terminal. Keep the **system Python** unchanged.
Linux	Install **Python 3** with your package manager. Do not replace the **system Python**.

Windows PATH fix. If `py` is missing, open **Edit environment variables for your account**, then **Path > Edit**. Add the installer's command directory, for example:

```
%LocalAppData%\Python\bin
```

macOS PATH fix. If the installer placed Python in `/usr/local/bin` but your terminal cannot find it, add that directory to your Z shell's **PATH**:

```
echo 'export PATH="/usr/local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For **Debian or Ubuntu**:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

Verify the installation and executable location:

```
# Windows
py --version
py -m pip --version
where py

# macOS or Linux
python3 --version
python3 -m pip --version
which -a python3
```

2.5 Route A: create an isolated virtual environment

venv keeps this project's packages separate from other projects. See the **venv documentation**.

The root-level **`requirements.txt`** lists dependencies, including PyTorch (`torch`) for Labs 7–8. A CPU is sufficient.

```
numpy
pandas
matplotlib
scikit-learn
joblib
jupyter
torch
```

Create the environment in the **project root**:

```
# Windows
py -m venv .venv

# macOS or Linux
python3 -m venv .venv
```

Activate it from the same folder:

```
# Windows (PowerShell)
.venv\Scripts\Activate.ps1

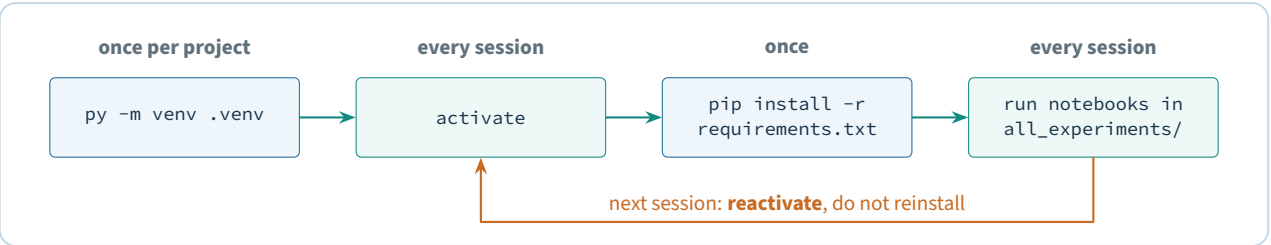
# macOS/Linux
source .venv/bin/activate
```

Install dependencies after activation:

```
# Windows
py -m pip install --upgrade pip
py -m pip install -r requirements.txt

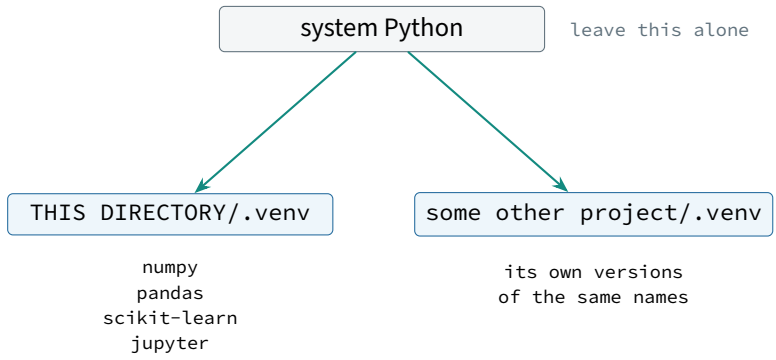
# macOS or Linux
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
```

Install once; activate every session. Reuse **`.venv`** without reinstalling unchanged requirements.



Key idea

Activate first, then install. Keep course packages inside the **project environment**, not in a shared global installation.



Separate environments, separate packages. Activate the intended **project environment** before running `pip install`.

2.6 Route B: Anaconda or Miniconda

Miniconda supplies Python and `conda`; **Anaconda** adds Navigator and preinstalled packages. Download **Miniconda** or **Anaconda**.

Windows Use default installer options without adding conda to PATH. Open **Anaconda Prompt** from Start.

macOS / Linux Run the `.pkg` or `.sh` installer, then close and reopen the terminal. The prompt shows `(base)`, which means conda is active.

Create one environment for the course, with a Python version the libraries support, then **activate** it:

```
conda create -n ml python=3.12
conda activate ml
```

Install dependencies with the same `requirements.txt` used in Route A:

```
pip install -r requirements.txt
```

Create once, activate each session, and avoid installing into `(base)`. Check the active environment and Python:

```
conda env list
python -c "import sys; print(sys.executable)"
```

Common mistake

Avoid installing the same package with both conda and pip. Use pip with the course requirements. Recreate a broken environment with `conda env remove -n ml`, then repeat setup.

Troubleshooting

Conda not found on Windows: use Anaconda Prompt, or initialize PowerShell there with `conda init powershell`. **Jupyter import failure:** activate `ml`, restart Jupyter, and check the kernel with Lab 1's environment cell.